

Cyber Security Lab Report — Part 1

PROJECT 1: PASSWORD STRENGTH ANALYZER

Objective: Develop a standalone Python script to evaluate password strength using regex pattern matching and Shannon bit-entropy calculation.

SOURCE CODE IMPLEMENTATION (PYCHARM STYLE)

```
import re
import math

def analyze_password(password):
    score = 0
    feedback = []

    length = len(password)
    if length < 8:
        feedback.append("Password is too short (min 8 chars required).")
    elif length >= 12:
        score += 2
    else:
        score += 1

    if re.search(r"[a-z]", password): score += 1
    else: feedback.append("Add lowercase letters.")

    if re.search(r"[A-Z]", password): score += 1
    else: feedback.append("Add uppercase letters.")

    if re.search(r"\d", password): score += 1
    else: feedback.append("Add numeric digits.")

    if re.search(r"[!@#$%^&*(),.?\":{}|<>]", password): score += 1
    else: feedback.append("Add special characters.")

    charset_size = 0
    if re.search(r"[a-z]", password): charset_size += 26
    if re.search(r"[A-Z]", password): charset_size += 26
    if re.search(r"\d", password): charset_size += 10
    if re.search(r"[!@#$%^&*(),.?\":{}|<>]", password): charset_size += 32

    entropy = length * math.log2(charset_size) if charset_size > 0 else 0

    if score >= 5 and entropy >= 50: rating = "Strong"
    elif score >= 3: rating = "Moderate"
    else: rating = "Weak"

    return {"score": score, "entropy": round(entropy, 2), "rating": rating, "feedback": feedback}
```

EXECUTION & TEST OUTPUT EXAMPLES

```
$ python3 password_analyzer.py

--- TEST CASE 1: WEAK PASSWORD ---
Input: "hello"
Result Rating : WEAK
Score         : 1/6
Bit Entropy   : 23.51 bits
Suggestions   :
- Password is too short (min 8 chars required).
- Add uppercase letters.
- Add numeric digits.
- Add special characters.

--- TEST CASE 2: MODERATE PASSWORD ---
Input: "User2026"
Result Rating : MODERATE
Score         : 4/6
Bit Entropy   : 47.63 bits
Suggestions   :
- Add special characters.
```

Cyber Security Lab Report — Part 1

PROJECT 2: DATA ENCRYPTION AND DECRYPTION TOOL

Objective: Implement symmetric AES-128 encryption and decryption using Python's standard `cryptography.fernet` library.

SOURCE CODE IMPLEMENTATION (PYCHARM STYLE)

```
from cryptography.fernet import Fernet

def generate_key():
    return Fernet.generate_key()

def encrypt_message(message: str, key: bytes) -> bytes:
    f = Fernet(key)
    return f.encrypt(message.encode('utf-8'))

def decrypt_message(encrypted_data: bytes, key: bytes) -> str:
    f = Fernet(key)
    return f.decrypt(encrypted_data).decode('utf-8')

if __name__ == "__main__":
    key = generate_key()
    msg = "Confidential Report Data 2026"

    ciphertext = encrypt_message(msg, key)
    plaintext = decrypt_message(ciphertext, key)
```

EXECUTION & TEST OUTPUT EXAMPLES

```
$ python3 encryption_tool.py

[+] Generating 256-bit AES Fernet Cryptographic Key...
    Key Output : b'xK9_mP2vL01zQ9wR7yT4uI8o03pA5sD6fG7hJ8kL9mA='

[+] Input Message Plaintext:
    "Confidential Report Data 2026"

[+] Executing Encryption...
    Ciphertext Output:
    b'gAAAAABmX_9vK8zQ1wR2yT3uI4o05pA6sD7fG8hJ9kL0mA1nB2cC3dE4fG5hJ6kL7mN8oP=='

[+] Executing Decryption with Matching Key...
    Decrypted Plaintext Output:
    "Confidential Report Data 2026"

[!] TEST FAILSAFE CASE: Decryption with invalid/wrong key
    Result: cryptography.fernet.InvalidToken
    Verification Success: Payload cannot be decrypted without the exact original key.
```

KEY OBSERVATIONS

- Fernet utilizes 128-bit AES in CBC mode with PKCS7 padding and HMAC authentication to prevent tampering.
- Ensures both data confidentiality and message integrity during storage or transmission.